ONE LANGUAGE UNA PLATAFORMA MULTICANAL PARA APOYAR LA COMUNICACION MEDIANTE LENGUA DE SEÑAS COLOMBIANA

Articulo tecnico del proyecto formativo ADSO-3145555

Equipo One Language | 4 de noviembre de 2026

Resumen

One Language es una plataforma educativa compuesta por cliente web, aplicacion movil, API backend, base de datos relacional y servicio de reconocimiento de señas. El cliente captura imagenes de cámara durante una sesión iniciada por la persona usuaria; MediaPipe extrae landmarks de las manos y un modelo BiLSTM clasifica secuencias temporales. El backend administra cuentas, permisos e historial, mientras PostgreSQL persiste los datos. La arquitectura separa inferencia y transacciones para facilitar despliegue y mantenimiento. Las pruebas locales recientes del backend y la base temporal pasaron; esos resultados no prueban precisión general del modelo. El sistema reconoce un vocabulario limitado de señas aisladas y requiere evaluación ampliada antes de considerarse una solución de traducción continua.

Palabras clave: accesibilidad, lengua de señas colombiana, landmarks, BiLSTM, Expo, API.

1. Introduccion

Las barreras de comunicación afectan la interacción cotidiana entre personas sordas y oyentes. One Language explora una herramienta de apoyo que convierte un conjunto acotado de señas en texto y, cuando está habilitado, voz. Su propósito es educativo: presentar una experiencia accesible y demostrar integración entre aplicaciones, servicios web, persistencia y aprendizaje automático. No reemplaza intérpretes ni garantiza comprensión de contextos lingüísticos completos.

2. Objetivo y alcance

El objetivo es implementar clientes web y móviles que permitan autenticar usuarios, iniciar reconocimiento desde cámara, consultar traducciones e historial y gestionar opciones de accesibilidad. El componente IA se enfoca en reconocer clases discretas entrenadas con secuencias de landmarks. El alcance conocido incluye las etiquetas Bien, Cómo estás, Gracias, Hola, Mal, Tú y Yo. La cobertura de estas etiquetas no representa el vocabulario total de LSC ni reconoce conversación continua, gramática, expresiones faciales o variaciones demográficas como un sistema completo.

3. Arquitectura de la solución

El cliente web usa React/Vite y el móvil React Native/Expo. Ambos consumen la API Spring Boot para identidad, reglas de negocio e historial. El cliente mantiene una conexión WebSocket con FastAPI para inferencia de baja latencia. PostgreSQL almacena entidades transaccionales y Liquibase versiona

cambios de esquema. En producción, Nginx termina TLS y enruta tráfico a servicios Docker publicados solo en loopback o red privada; PostgreSQL no se expone al navegador.

Separar la inferencia del backend evita acoplar las dependencias de visión artificial al servicio transaccional. La API conserva la relación entre una traducción y la cuenta; la IA se limita a producir predicciones y no consulta directamente PostgreSQL.

4. Procesamiento de señas

Durante una sesión autorizada, la cámara aporta frames. MediaPipe localiza hasta dos manos y representa cada mano mediante 21 puntos. La secuencia temporal, de longitud configurada en 32 frames, se prepara y se entrega al clasificador recurrente bidireccional (BiLSTM). La salida se asocia con una etiqueta y una confianza; parámetros de estabilidad y umbrales reducen cambios espurios entre frames. El cliente presenta la predicción y la aplicación puede guardar la traducción en el backend.

El procesamiento usa coordenadas numéricas derivadas de los frames. La disponibilidad de cámara y envío de datos depende de la acción y permisos de la persona. El diseño no debe interpretarse como almacenamiento automático de video completo por el servicio IA.

5. Tecnologias

Web: React, Vite, Axios y Nginx. Móvil: React Native, Expo y Expo Router. Backend: Java 17, Spring Boot, Spring Security, validación, JPA, JWT y Liquibase. IA: Python, FastAPI, MediaPipe y TensorFlow/Keras. Persistencia: PostgreSQL 15 y volumen Docker. El transporte externo utiliza HTTPS/WSS.

6. Verificacion

El 4 de noviembre de 2026 se ejecutaron 41 pruebas del backend con cero fallos y cero errores; seis pruebas E2E/concurrencia se encontraban excluidas en la ejecución ordinaria. Al activar la configuración de integración contra PostgreSQL temporal, una prueba E2E y cinco de concurrencia pasaron. El flujo HTTP verificó registro 201, login 200, perfil autenticado 200 y perfil sin token 401; la contraseña se almacenó con BCrypt. Liquibase validó y aplicó 21 changesets sobre la base temporal.

Estas verificaciones miden comportamiento de API, base de datos y concurrencia bajo el conjunto local disponible. No miden exactitud, robustez, sesgo ni generalización del modelo visual. Para afirmar desempeño de IA se requiere un conjunto de prueba independiente, métricas por clase, intervalos de confianza, diversidad de participantes y condiciones de captura documentadas.

7. Limitaciones y trabajo futuro

El vocabulario es pequeño, por lo que el sistema no traduce cualquier oración en LSC. Iluminación, distancia, encuadre, velocidad, oclusión y diferencias individuales pueden afectar la detección. La inferencia necesita conectividad con el servicio remoto; una desconexión puede interrumpir la traducción. Se debe medir precisión y latencia en dispositivos reales, ampliar y documentar datos con consentimiento, evaluar equidad, mejorar manejo de errores y validar seguridad del WebSocket. También se recomienda automatizar CI contra PostgreSQL y completar controles de operación y rotación de secretos con responsables autorizados.

8. Conclusion

One Language integra aplicaciones multiplataforma, backend, PostgreSQL y reconocimiento de secuencias para explorar un apoyo educativo a la comunicación. La separación de servicios permite evolucionar cada parte de forma independiente, mientras las pruebas recientes respaldan algunos flujos de backend y persistencia. El aporte actual es un prototipo funcional de alcance acotado, no un traductor universal de LSC. La evolución responsable requiere más datos, evaluación formal del modelo, seguridad verificable y pruebas en contexto real.

Referencias

Spring Boot. Reference Documentation. https://docs.spring.io/spring-boot/reference/

Expo. Create a project. https://docs.expo.dev/get-started/create-a-project/

FastAPI. WebSockets reference. https://fastapi.tiangolo.com/reference/websockets/

PostgreSQL Global Development Group. PostgreSQL 15 Documentation. https://www.postgresql.org/docs/15/